CSE 4531: E-Commerce and Web Security

Quiz: 1

Duration: 30 minutes Total Marks: 15 Date: 22.12.25

Answer all three (3) questions. Figures inside the boxes in the right margin indicate marks of each question.

1. For each of the following security goals—**Confidentiality**, **Integrity**, and **Availability**—do the following: 3

1. Describe a **realistic scenario** in which the security goal is compromised in an e-commerce system.
2. Explain the **technical mechanism or type of attack** that leads to the compromise.
3. Discuss the **impact** of the compromise on the system, stored data, and end users.

✅ Model Answer:

Security Goal	Example Scenario (What happens?)	Attack Type (How it happens?)	Main Impact (Why is it harmful?)
Confidentiality	Hackers break into the database and **steal customer personal and payment data**	**SQL Injection** — attackers insert malicious database queries through input fields	**Privacy breach, financial fraud risk, and legal consequences for the company**
Integrity	An attacker **changes product prices or edits transaction records**	**Cross-Site Scripting (XSS)** — malicious script runs in the user's browser and modifies data	**Incorrect data, financial losses, and loss of trust because records become unreliable**
Availability	The website becomes **slow or completely unavailable to users**	**Distributed Denial-of-Service (DDoS)** — attacker floods the server with fake traffic	**Customers cannot shop, sales drop, and users become frustrated**

2 The ride-sharing application provides drivers with a profile update form through which they can modify **their own vehicle description** (illustrated in Figure 1.1). 6

.

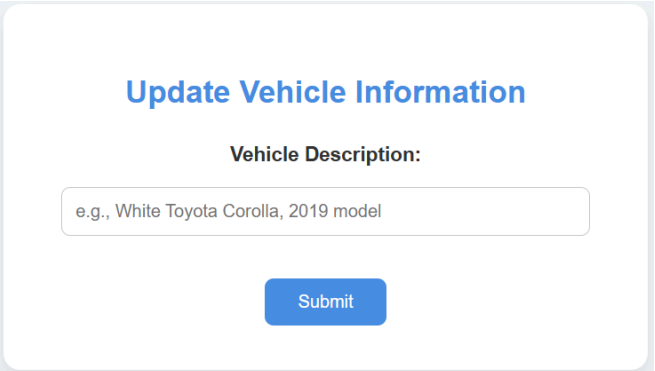

Figure 1.1: Profile update form used by drivers to edit vehicle details (entry point for user input).

When the form is submitted, the backend executes the following SQL query:

```
UPDATE drivers
SET vehicle_info = '$vehicle'
WHERE driver_id = $did;
```

Where:

- `$vehicle` contains the vehicle description entered by the driver through the form.
- `$did` is the authenticated driver's ID obtained from the session.

Only the `vehicle_info` field is intended to be editable through this form.

The `drivers` table contains a column named `rating`, which drivers are **not authorized** to modify. For example, **Alice**, a legitimate driver, currently has a rating of **4.8**.

Assume that:

- User input is **not sanitized**.
- Prepared statements are **not used**.
- **Eve**, a malicious driver, knows Alice's `driver_id which is "DID042"`.

Eve intends to **reduce Alice's rating from 4.8 to 1.2**. Demonstrate an SQL injection attack by specifying the input **Eve could enter in the Vehicle Description field** to modify Alice's rating.

Your answer must clearly specify the **injected payload** (5)

✅ Model Answer:

🔹 Example Injection Payload

Injected value for `vehicle_info`:

```
Nice car', rating = 1 WHERE driver_id = 42 #
```

(Assume `42` is the `driver_id` of the target driver.)

🔹 Resulting SQL Statement After Injection

```
UPDATE drivers
SET vehicle_info = 'Nice car', rating = 1
WHERE driver_id = 42 # '
WHERE driver_id = 17;
```

> Everything after `#` is treated as a **comment** and ignored by MySQL.

3 A trusted e-commerce website allows customers to submit product reviews. These reviews are stored in the database and later displayed on product pages **exactly as submitted**.

Alice, a legitimate customer, is authenticated while browsing product pages. Eve, a malicious user, notices that **JavaScript code included in a product review is executed in the browsers of users who view the affected product page**.

Answer the following:

(a) Injection and Storage of Malicious Code (2.5 marks)

- Explain how Eve can inject malicious code.
- Explain why the application stores the malicious code.

(b) Execution of Malicious Script in Alice's Browser (2 marks)

- Describe, step by step, how the injected script executes in Alice's browser.

(c) Session Capture (1.5 marks)

- Explain how Alice's session information can be captured.

✅ Model Answer:

(a) Injection & Storage (2.5 marks)

- **Injection:** Eve writes JavaScript inside a review (e.g., `<script>…</script>`).
- **Reason it's stored:** The site **does not sanitize/escape input**, so the script is saved in the database exactly as entered.

(b) Execution in Alice's Browser (2 marks)

1. Alice logs in and opens the product page.
2. The page loads reviews from the database.
3. Eve's script is included in the HTML.
4. Alice's browser executes it as normal page JavaScript (stored XSS).

(c) Session Capture (1.5 marks)

- The script reads Alice's **session cookie/token** (e.g., via `document.cookie`)
- Sends it to Eve's server
- Eve uses it to **hijack Alice's session**.